

予選 解説

2006年12月22日・情報オリンピック日本委員会

第6回日本情報オリンピック 予選

問題 1 得点

配点 20点

時間制限 10 sec

メモリ制限 256 MiB

解説

この問題は、肩慣らしと JOI 予選の出題形式に慣れてもらうことを意図して出題されている。入力は 2 行で、それぞれの行に 0 以上 100 以下の整数が空白を区切りとして 4 つ含まれている。つまり、入力のは数は 8 で固定である。よって、繰り返しを用いる必要はなく、以下の処理を行えばよい。

1. 入力の 1 行目の 4 つの整数を読み込み、それらの和 S を計算
2. 入力の 2 行目の 4 つの整数を読み込み、それらの和 T を計算
3. S と T の大小を比較し小さくない方を出力

注：「小さくない方」という表現は、 S と T が等しくない場合は「大きい方」を意味し、 S と T が等しい場合は「どちらでもよい」ことを意味している。

この程度の処理を行う必要が実際に生じた場合、プログラムを作成するまでもないだろう。

原文：https://www2.ioi-jp.org/joi/2006/2007-yo-prob_and_sol/2007-yo-review/2007-yo-t1-review.html

問題 2 未提出者は誰だ

配点 20点

時間制限 10 sec

メモリ制限 256 MiB

解説

入力は 28 個の重複しない 1 以上 30 以下の整数である。出席番号ごとに提出したかどうかを記憶するサイズ 30 の配列を 1 つ用意して（この配列の名前を F とする），以下のような単純な処理を行ったので十分であろう。

1. F の要素すべてを，「未提出」で初期化する。
2. i を 1 で初期化し，次を 28 回繰り返す：
 - 入力の i 行目の整数 x_i を読み込み，配列 F の x_i に対応する要素を「提出」にする。
 - i を 1 増加させる。
3. i を 1 で初期化し，次を 30 回繰り返す：
 - F の i 番目の要素が「未提出」なら i を出力する。
 - i を 1 増加させる。

「提出」と「未提出」を表すのに，論理型を扱えるプログラミング言語であれば「真」と「偽」に対応する論理型定数を用いてもよいし，論理型を備えていないプログラミング言語では整数型定数の 1 と 0 を用いてもよい。

原文：https://www2.ioi-jp.org/joi/2006/2007-yo-prob_and_sol/2007-yo-review/2007-yo-t2-review.html

問題 3 シーザー暗号

配点 20点

時間制限 10 sec

メモリ制限 256 MiB

解説

大文字のアルファベットを次のように 0 から 25 の整数に対応づける．

文字	A	B	C	D	E	F	G	H	I	J	K	L	M	N	O	P	Q	R	S	T	U	V	W	X	Y	Z
整数	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21	22	23	24	25

すると、この問いの変換は、0 から 22 までは 3 を加えるという操作に、23, 24, 25 は 3 を加えてから 26 を引くという操作に対応する．（この 2 つの場合をまとめて、3 を加えて 26 で割った余りを求める操作に対応すると考えることもできる．）

変換前	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21	22	23	24	25
変換後	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21	22	23	24	25	0	1	2

よって逆の変換は、0, 1, 2 は 26 を加えてから 3 を引くという操作に対応し、3 から 25 は 3 を引くという操作に対応する．（この 2 つの場合をまとめて、23 を加えて 26 で割った余りを求める操作に対応すると考えることもできる．）

逆変換前	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21	22	23	24	25
逆変換後	23	24	25	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21	22

よって、上記の対応に従いアルファベットを整数に変換し、逆の変換を行い、再び上記の対応に従い整数をアルファベットに戻せばよい．例えば、変換後の文字列 EBH は、次のように文字列 BYE に戻る．

E → 4 → 1 → B
 B → 1 → 24 → Y
 H → 7 → 4 → E

他に、文字の対応表を配列などで実現し参照するという方法などが考えられる．

原文：https://www2.ioi-jp.org/joi/2006/2007-yo-prob_and_sol/2007-yo-review/2007-yo-t3-review.html

問題 4 カードの並び替え

配点 20点

時間制限 10 sec

メモリ制限 256 MiB

解説

$2n$ 枚のカードが、上からどのような順番で並んでいるかを配列に保存し、その配列を更新し続ければ良い。
ここでは、「カードの並びの保存の仕方」「 k でカットの実行の仕方」「リフルシャッフルの実行の仕方」について解説を行う。

データの保存の仕方

「操作を行う前のカードの並びを蓄える配列」と「操作を行った後のカードの並びを蓄える配列」の 2 つを準備すると良いだろう。

サイズ $2n$ の 1 次元配列を (2 つ) 準備し、そこにカードの並びを保存すればよい。たとえば、カードが上から 3, 5, 1, 4, 2 の順に並んでいるとしたら、配列に 3, 5, 1, 4, 2 の順番に値を保存する。

一番最初は、上から 1, 2, 3, $\dots$, $2n$ の順に積み重なっているので、配列に保存されているデータも、順に 1, 2, 3, $\dots$, $2n$ となる。

k でカットの実行の仕方

まず、「操作を行う前のカードの並びを保存している配列」の $k+1$ 番目から $2n$ 番目までの値を、「操作を行った後のカードの並びを保存している配列」の 1 番目から $2n-k$ 番目までにコピーする。

その後、「操作を行う前のカードの並びを保存している配列」の 1 番目から k 番目までの値を、「操作を行った後のカードの並びを保存している配列」の $2n-k+1$ 番目から $2n$ 番目までにコピーすればよい。

最後に、「操作を行った後のカードの並びを保存している配列」の内容を、「操作を行う前のカードの並びを保存している配列」にコピーしておく。

リフルシャッフルの実行の仕方

「操作を行った後のカードの並びを保存している配列」の j 番目には、

- j が奇数ならば、「操作を行う前のカードの並びを保存している配列」の $(j+1)/2$ 番目の値を、
- j が偶数ならば、「操作を行う前のカードの並びを保存している配列」の $n+j/2$ 番目の値を、

コピーするようにすればよい。

最後に、「操作を行った後のカードの並びを保存している配列」の内容を、「操作を行う前のカードの並びを保存している配列」にコピーしておく。

プログラム全体の処理の流れ

1. 入力ファイルから値 n を読み込む。
2. サイズ $2n$ の配列を 2 つ宣言する。
3. 「操作を行う前のカードの並びを保存している配列」が, $1, 2, 3, \dots, 2n$ の順番に値を持つように初期化する。
4. 入力ファイルから操作の回数 m を読み込む。
5. 次を m 回繰り返す。
 1. 入力ファイルからカードを並べ替える方法を読み込む。
 2. 読み込んだ値に従って, リフルシャッフルもしくはカットを行う。
6. 答えとして「操作を行う前のカードの並びを保存している配列」を出力する。

となる。

編集注：公式解説ページには, カット後のコピー先の終端およびリフルシャッフルの配列参照に明らかな表記上の不整合がある。ここでは公式問題文に記された操作と一致するよう, コピー先を $2n-k+1 \dots 2n$ とし, 操作後の j 番目へコピーする元の位置を示す形に補正した。

原文：https://www2.ioi-jp.org/joi/2006/2007-yo-prob_and_sol/2007-yo-review/2007-yo-t4-review.html

問題 5 品質検査

配点 20点

時間制限 10 sec

メモリ制限 256 MiB

解説

ある三つの部品をつないで検査して、「合格」の結果が出ていれば、その三つの部品はすべて正常だとわかる。言い換えれば、「合格」の結果が出た検査で一度でも使われていた部品はすべて正常であるとわかる。逆に、正常であるとわかる部品はこれらの部品だけである。というのは、「合格」の結果が出た検査で一度も使われなかった部品は、すべて故障しているとしても検査結果と矛盾しないからである。

故障しているとわかる部品はどのような部品だろうか？ 正常とわかる部品二つと正常かどうかが不明な部品一つをつないで検査して、「不合格」の結果が出れば、不明だった部品は故障しているとわかる。逆に、部品 x を正常とわかる部品二つとつないで検査したことがなければ、 x が正常であるとしても検査結果と矛盾しない（例えば、正常とわかる部品と x だけが正常で、他の部品がすべて故障しているという場合を考えればよい）。よって、そのような部品 x は故障しているとも正常であるとも決まらない。

まとめると、すべての部品を故障しているとわかるもの、正常だとわかるもの、どちらとも決まらないものの3種類に分類するには、次のようにすればよい。

1. まず、各部品について「故障」「正常」「不明」のいずれかを記録しておくための場所を用意する。最初はすべて「不明」に初期化しておく。
2. 次に、検査結果のうち「合格」を表す行について、検査に使われた三つの部品を「正常」とマークする。
3. 最後に、検査結果のうち「不合格」を表す行について、検査に使われた三つの部品のうち二つが「正常」であれば、残りの一つを「故障」とマークする（そうでないときは無視する）。

他の解法として、すべての部品の「故障」「正常」の組合せ 2^{a+b+c} 通りについて、検査結果と矛盾しないかどうかを調べるという方法がある。この方法は、部品の個数 $a+b+c$ が大きくなると、調べる組合せの個数が急激に増えるため、 $a+b+c$ が小さい場合しか現実的に使えない。この方法では入力 1 のみが解ける。

このほか、上で述べた効率の良い解法と効率の悪い解法の間にあたるような解法も考えられるが、そのような解法には相応に部分点が与えられることを意図して入力を決めた。

原文：https://www2.ioi-jp.org/joi/2006/2007-yo-prob_and_sol/2007-yo-review/2007-yo-t5-review.html

問題 6 通学経路

配点 20点

時間制限 10 sec

メモリ制限 256 MiB

解説

最短経路の個数を数える問題である。同様の問題を数学の授業でやったことのある人も多いだろう。「計算の仕方は分かったが、うまくプログラムが書けなかった」という人も多かったのではないだろうか。

この問題の効率の良い解法は次のようなものである。交差点 $(1, 1)$ から出発して、東または北にのみ向かって移動し、交差点 (j, k) に移動する方法を $t[j][k]$ 通りとおく。以下では簡単のため、 j, k は $0 \leq j \leq a, 0 \leq k \leq b$ の範囲であるとし、 j または k が 0 のときは $t[j][k] = 0$ とおく。このとき、 $t[j][k]$ は次の漸化式をみたす。

$(1, 1)$ が工事中でないとき： $t[1][1] = 1$
 j または k が 0 のとき： $t[j][k] = 0$
交差点 (j, k) が工事中のとき： $t[j][k] = 0$
そうでないとき： $t[j][k] = t[j-1][k] + t[j][k-1]$

この漸化式を用いて $t[a][b]$ を求めればよい。

解法 1

2 重ループを使って、 $t[j][k]$ を j, k が小さい方から順に計算する。具体的には、

- 配列 $t[j][k]$ ($0 \leq j \leq a, 0 \leq k \leq b$) を全て 0 に初期化する。
- j を 1 から a まで動かす。
 - k を 1 から b まで動かす。
 - $(j, k) = (1, 1)$ であり、さらに $(1, 1)$ が工事中でないとき： $t[j][k] = 1$
 - (j, k) が工事中のとき： $t[j][k] = 0$
 - そうでないとき： $t[j][k] = t[j-1][k] + t[j][k-1]$

により、配列 $t[j][k]$ が全て計算できる。その後、 $t[a][b]$ を出力する。

解法 2

関数の中で自分自身を呼び出す「再帰」のテクニックを使ってもよい。次のような関数 `func(j, k)` を用意し、`func(a, b)` を出力する。

```
func(j, k)
{
    j または k が 0 のときは 0 を返す.
    (j, k) が工事中の時は 0 を返す.
    (j, k) = (1, 1) なら 1 を返す.
    そうでないときは, func(j-1, k) + func(j, k-1) を返す.
}
```

なお、この関数 `func(j, k)` は、このままでは同じ (j, k) に関する計算を 2 回以上行う可能性があるため、効率が悪い。一度計算した結果を記憶領域に格納しておき、二度目以降はそれを参照する、という工夫によって高速化することもできる。

漸化式の問題を「再帰」を使って解く利点の一つは、再帰呼び出しの過程で、目的地 (a, b) に到達する可能性のある $t[j][k]$ だけが計算されることにある（一方、上に述べた 2 重ループの方法では、全ての $t[j][k]$ を計算することになるので、無駄な計算が含まれてしまう）。「再帰」は、うまく使うと短い行数のプログラムで効率よく計算することができる強力なテクニックなので、ぜひこれを機会に使いこなせるようになってほしい。

解法 3

さらに別解として、かなり効率は悪いが、「全ての経路を列挙して調べる」という解法（全数検査法）もある。プログラム内で $C(a+b-2, a-1)$ 通りの経路を全て生成し、工事中の交差点を通らないものの個数を数えればよい。

このように、この問題には複数の解法がある。もちろん、それぞれの解法には、短所・長所がある。今回は、入力データのサイズをかなり小さく設定したので $(1 \leq a, b \leq 16)$ 、正しく動作するプログラムさえ書ければ、どのような解法でも満点を取ることは十分に可能である。もちろん、JOI 本選や IOI で出題されるような難易度の高い問題では、問題の状況にあわせて最適なアルゴリズムを考え、プログラムを書くことも必要になってくる。全数検査法では、データのサイズが大きい場合に制限時間内に答えを出力できずに、部分点しか得られないこともあるだろう。

この問題ができた人も、今回はうまくいかなかった人も、ぜひ、上に述べた様々な解法をきちんと理解して、もう一度、自分の手でじっくりとプログラムを書いて解き直してみしてほしい。そして、そこで使われる考え方やテクニックをしっかりと身に付けて、より難易度の高い問題にも対応できる実力をつけてほしい。

原文：https://www2.ioi-jp.org/joi/2006/2007-yo-prob_and_sol/2007-yo-review/2007-yo-t6-review.html

出典：情報オリンピック日本委員会「第6回日本情報オリンピック JOI 2006/2007 予選 問題・データ・解説」。本PDFは各解説ページを問題 1→2→3→4→5→6 の順に再構成したものです。

https://www2.ioi-jp.org/joi/2006/2007-yo-prob_and_sol/index.html

公式アーカイブでは、第5回以降の予選問題について CC BY-SA 4.0 の利用条件が案内されています。配点は原大会の成績集計（各問題20点）に合わせ、時間・メモリ制限は AtCoder の JOI 過去問表示を参照しました。公式ページには、問題4の公開入力データ末尾に無関係な1行が混入している可能性がある旨の追記（2015年2月21日）があります。